Table of Contents

Ripple Carry Adder 2
- Generate-Propagate Logic 4
  - Manchester Carry Chain 5

Carry Bypass Adder 6

Carry Select Adder 8

Carry Lookahead Adder 9
- Group Generates and Propagates 10
  - Dot Operator 10

Kogge Stone Adder 11
- Advantages of Kogge Stone Adder 13
- Disadvantages of Kogge Stone Adder 13

Brent Kung Adder 13
- Advantages of Brent Kung Adder 13
- Disadvantages of Brent Kung Adder 13

Summary 14

In this lecture, we will talk about adders. Why? Because arithmetic circuits are the bulk of most ASICs and usually the critical path passes through arithmetic circuits, mainly adders and multipliers.

For an N-bit adder, the output is N+1 bits.

What we will do now is that we will study many types of adders and compare between them.

Ripple Carry Adder

The building blocks of any adder are one half adder for the LSB and full adders for the rest of the bit positions. However, we sometimes use adders to build larger adders, so we will use a full adder even with the LSB.

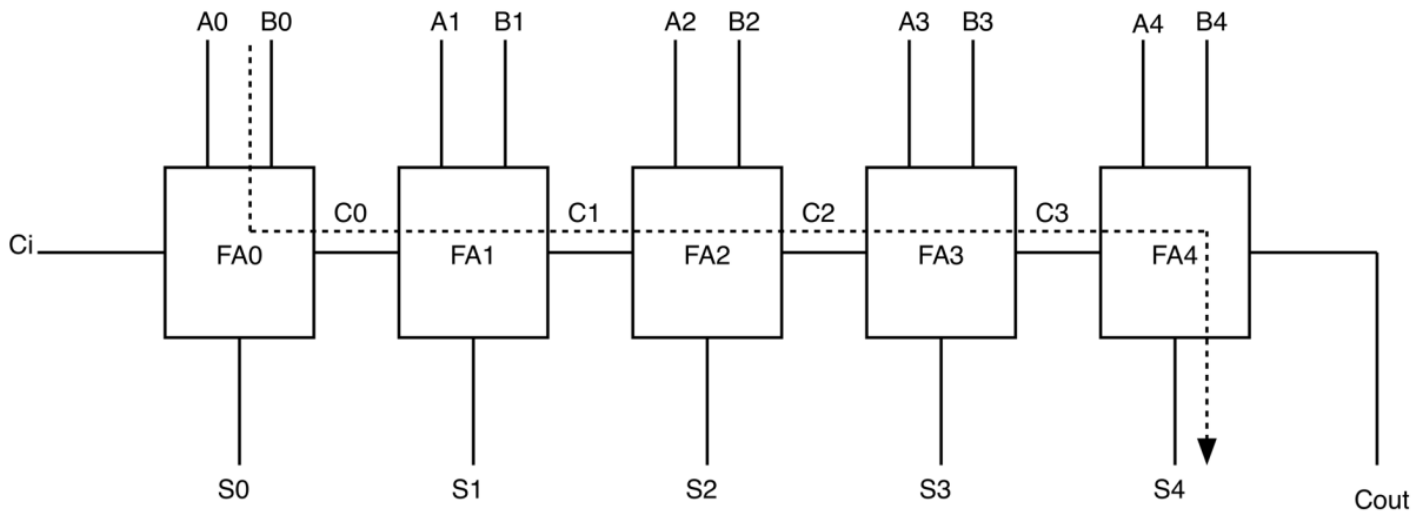
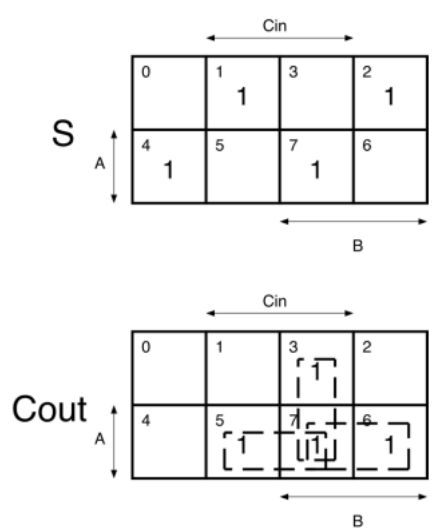


Figure 1. Forming a 5-Bit Adder Using Full Adders

A, B, Cin	S	Cout
000’	0	0
001’	1	0
010’	1	0
011’	0	1
100’	1	0
101’	0	1
110’	0	1
111’	1	1



Thus:

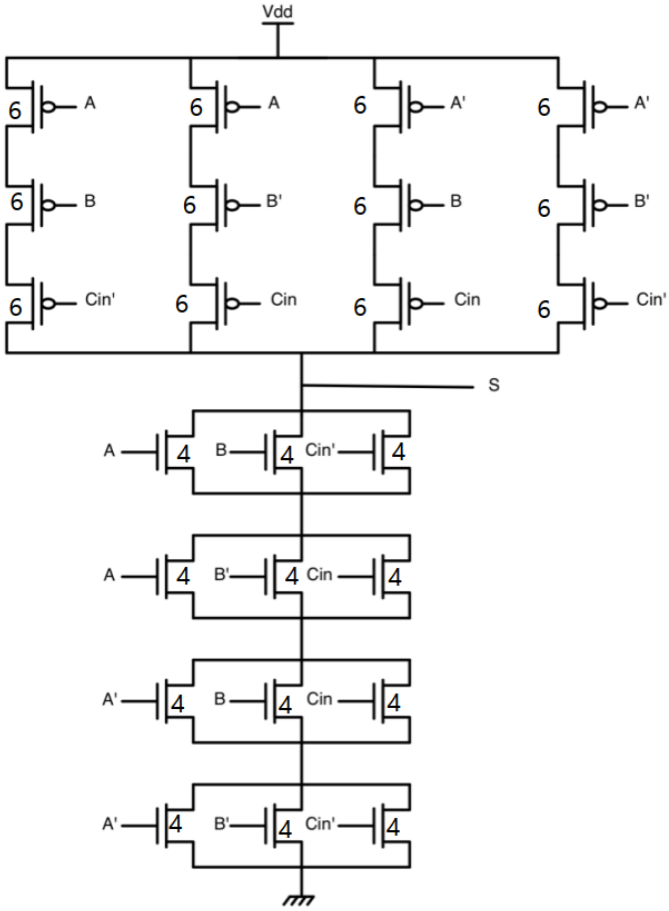
$$S = A \oplus B \oplus C_{in} \quad = AB'C_{in}{}' + A'B'C_{in} + ABC_{in} + A'BC_{in}{}'$$

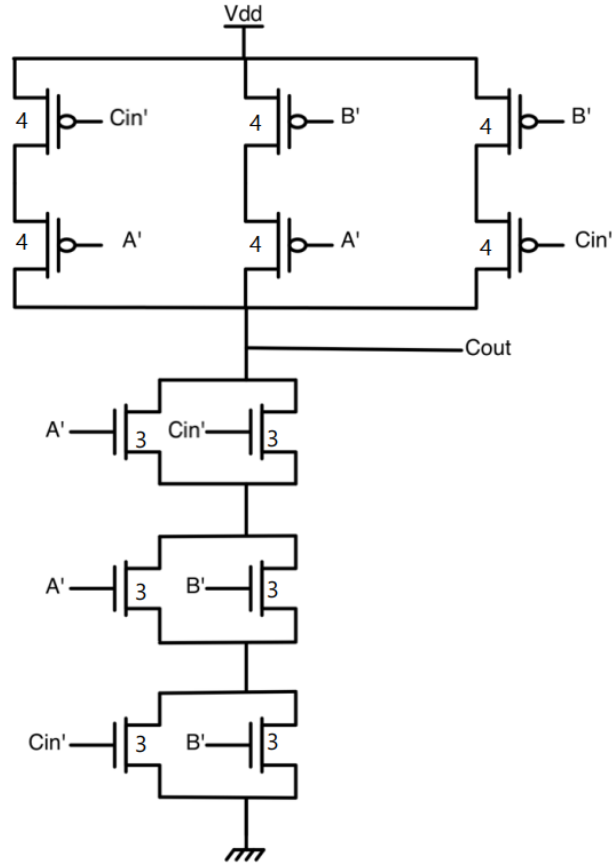
$$C_{out} = AC_{in} + AB + BC_{in}$$

We know that to implement S and C_{out}, we need to take common factors to simplify the expressions and hence minimize the number of transistors, but we will trivially use the expressions as they are:

$$S' = (A' + B + C_{in})(A + B + C_{in}{}')(A' + B' + C_{in}{}')(A + B' + C_{in})$$

$$C'_{out} = (A' + C_{in}{}')(A' + B')(B' + C_{in}{}')$$



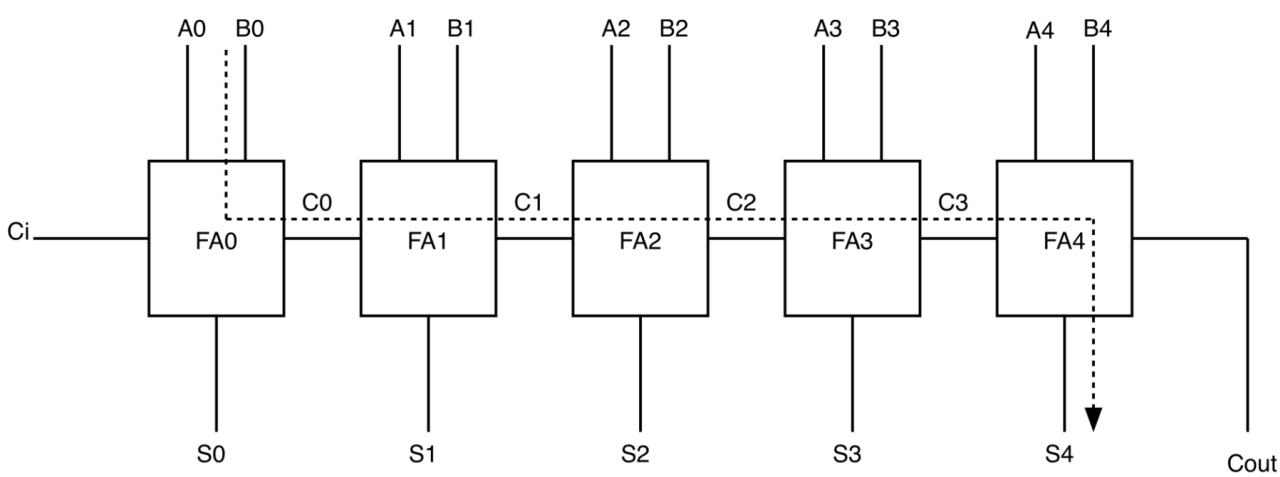


$$t_p|_S = 0.69 * 36R_oC_o$$

$$t_p|_{C_{out}} = 0.69 * 18R_oC_o$$

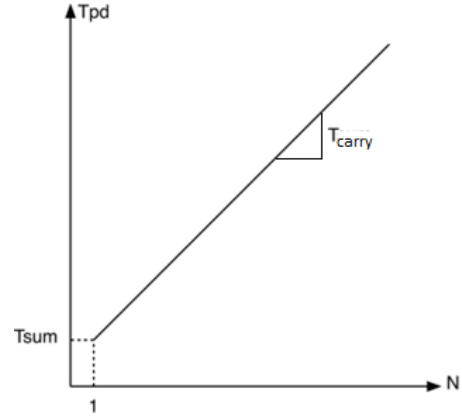
The question is: what is the delay of this type of adders?

To evaluate the delay, we need to identify the critical path. Since sum has larger delay than C_{out}, then the critical path is from any input of the first full adder to the sum output of the last full adder as shown below:



Thus, in general:

$$\boldsymbol{T_{add} = (N-1)T_{carry} + T_{sum}}$$



Notes:

- Although $T_{sum} > T_{carry}$, T_{carry} is much more important than T_{sum} since it is multiplied by $(N-1)$.
- Delay grows *linearly* with the number of bits of the operands. In other words, delay growth is $O(N)$.

Eh a7san delay growth 3ayzeen newsalo? el delay growth momken yeb2a linear, quadratic, square root, or logarithmic. Akeed a7san 7aga nefsena newsalaha heya el logarithmic growth.

Generate-Propagate Logic

El truth table bta3 el full adder shown t7t on the left 3ady 5ales. Hanzawed 3 columns on the right:

- G: Generate.
- D: Delete.
- P: Propagate.

Eh dool? el nas 5adet balha men 3 7agat fel truth table bta3 el full adder:

1. Fi awel two rows, nela7ez en C_{out} be zero irrespective of the value of C_{in}. Thus, el C_{out} hena yo3tabar "deleted", w bel taly el D output da hayeb2a be 1.
2. Fi a5er two rows, nela7ez en C_{out} be one irrespective of the value of C_{in}. Thus, el C_{out} hena yo3tabar "generated", w bel taly el G output da hayeb2a be 1.
3. Fel four rows el fel nos, nela7ez en C_{out} beya5od el value bta3t C_{in}. Thus, el C_{in} hena yo3tabar bey "propagate" lel C_{out}, w bel taly el P output da hayeb2a be 1.

A	B	Cin	Cout	S	G	D	P
0	0	0	0	0	0	1	0
0	0	1	0	1	0	1	0
0	1	0	0	1	0	0	1
0	1	1	1	0	0	0	1
1	0	0	0	1	0	0	1
1	0	1	1	0	0	0	1
1	1	0	1	0	1	0	0
1	1	1	1	1	1	0	0

Notice that the three outputs are mutually exclusive, so each row in the truth table is either a generate, propagate, or a delete row.

$$G = AB$$

$$P = A \oplus B$$

$$D = (A + B)'$$

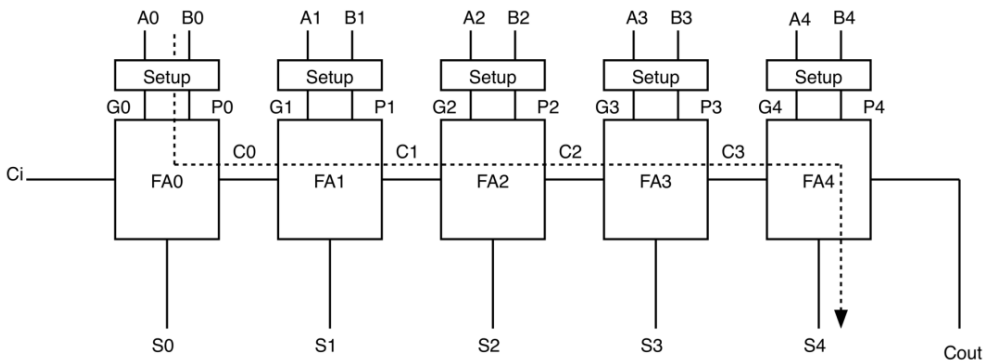
Tyb el S wel C_{out} in terms of G, P, D hayeb2o given by eh?

$$S = P \oplus C_{in}$$

$$C_{out} = G + P.C_{in}$$

Fa nela7ez enohom mesh function fel delete D logic, fa el D da mesh mohem bel nesbalna, el mohem G w P. 3ashan keda el title beta3 el section da "generate-propagate logic", mafehosh delete. Fa from now on hana3taber en el inputs el da5la 3al full adders heya P, G, and C_{in} instead of A, B, and C_{in}. Hal e7na keda 2alelna el delay bta3 S aw C_{out}? fel 7a2e2a la2, lw garabt t3awad fel 2 expressions el fato dool 3an P w G bel expression bot3hom, hatla2y en S w C_{out} reg3o bzbt zy el case beta3t el full adder 3ady 5ales. Omal eh el estefada men el generate-propagate logic da? just haysahelo 3alena fehm 7agat odaam, laken homa ma2alelosh el delay wala bta3 el sum wala bta3 el carry.

Fih meza qaweya fel G & P w heya enohom mesh function fel C_{in}. Fa e7na lw 3andena ripple carry adder, fa kol el full adders ye2daro ye7sebo el G & P bto3hom in parallel men el A wel B el da5leen 3leehom, mesh motareen enohom yestano C_{in} teg-haz men el full adders el ablehom. Fa7na momken n7ot logic 3and el inputs bta3t kol el full adders ye7sebo el G & P 3la tool, w nesamy el logic da setup logic masalan, w yeb2a keda 3amalna "Ripple carry adder with G-P logic" as shown below:



W ne5aly balna en el full adders delwa2ty bey implement-o el two equations dool (i.e. mesh zy abl keda):

$$S = P \oplus C_{in}$$

$$C_{out} = G + P.C_{in}$$

Fa l7d delwa2ty keda mafeesh ay estefada zy ma olna w el critical path beta3 el adder deh el delay beta3o:

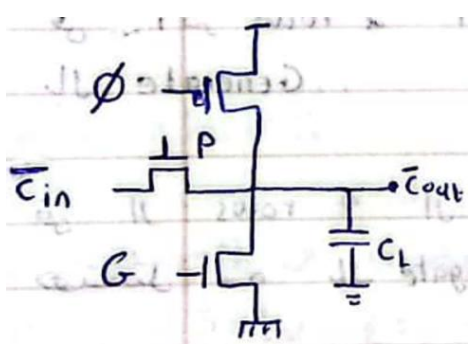
$$T_{add} = T_{gp} + (N-1)T_{carry} + T_{sum}$$

where:

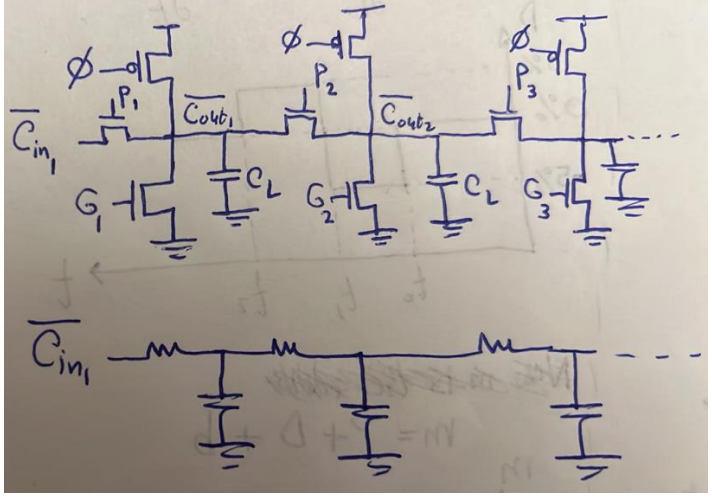
- T_{gp}: delay needed to calculate G and P, which will be the xor gate needed to calculate P.
- $T_{carry} = G + PC_{in}$.
- $T_{sum} = P \oplus C_{in}$.

Manchester Carry Chain

De just tare2a tanya en badal ma ne7seb $C_{out} = G + PC_{in}$ using static gates, it uses dynamic logic (since they are faster) as shown below:



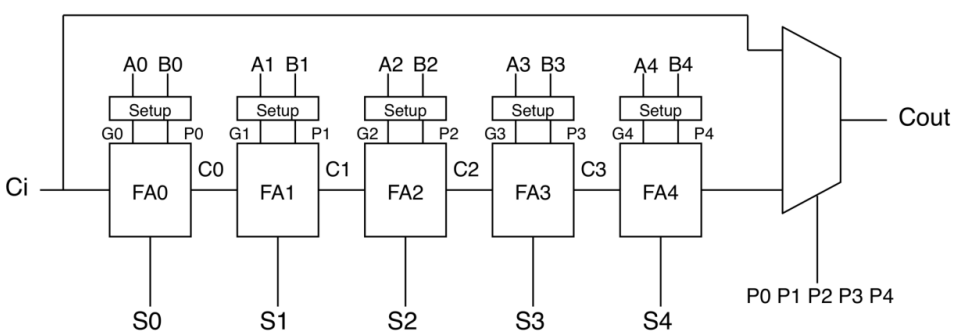
Moshkelet el implementation da hatezhar lama nebda2 ne cascade kaza full adder wara b3d, which will obviously happen, en hayezharlena RC ladder taweel as shown below, w e7na 3arfeen men Elmore model en el delay bta3 el RC ladder da beyzeed quadratically ma3 el number of stages.



Tyb eh el 7al? enena nefsel been kol stage wel tanya be buffer (i.e. two static inverters).

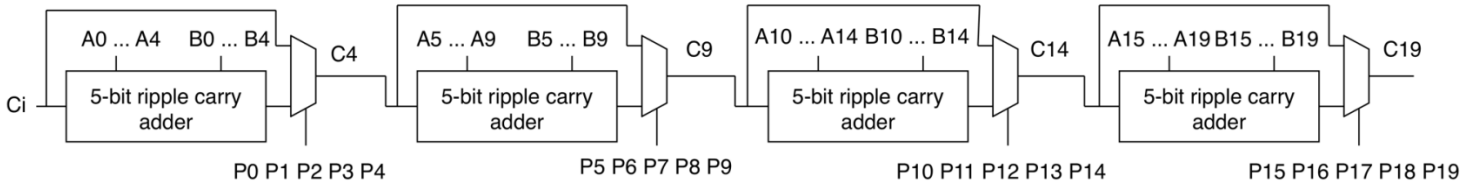
Carry Bypass Adder

Da awel noo3 fe3lan ne2dar nastafeed fih bel generate-propagate logic. Shown below is a ripple carry adder 3adeya 5ales, bs zawedna 3leeha mux fel a5er, fa b2a esmaha carry bypass adder. El mux da by3ml eh? bey2ol walahy lw ana la2eet en kol el stages bet propagate, fa da ma3nah en el final C_{out} hayeb2a howa el C_{in}, w bel taly mesh me7tag eny a7seb el C_{out} through el ripple carry chain el t7t w ha bypass el C_{in} 3la tool 3la bara men el path el foo2. Fa keda b2a 3andy ripple carry chain path w bypass path, w keda hayeb2a el delay bta3 el adder fel case bta3t el bypass path de $t_{gp} + T_{mux}$ bs. 5aly balak enta hates2al nafsak "ezay el delay bta3 el adder howa $t_{gp} + T_{mux}$, leeh mesh el delay bta3 S_4? ". Mashy enta m3ak 7a2 f3ln en fel case bta3t el bypass el S_4 leha delay akbar men el C_{out}, w actually, heya S_4 dayman aslan leha delay akbar men el C_{out}. However, e7na dayman benrakez ma3 el C_{out} la2en e7na mesh hanesta5dem el adder el t7t de lwa7daha keda, e7na lesa han7otaha gamb stages zayaha tanyeen, fa el C_{out} da hay "ripple" through many other stages, be ma3na en 7ata fi 7alet el bypass f3ln el S_4 haya5od $4T_{carry} + T_{sum}$ 3ashan yet7eseb, w laken el concern bel nesbalna hayeb2a still howa el $t_{gp} + T_{mux}$ bta3t el C_{out} la2en lesa el C_{out} da hay5osh 3la stage ba3deya, fa aslan 3o2bal ma el stages de t5ls haykoon el S_4 et7asab men badry, fa el sum mesh howa 5ales el azma.



W bel taly el adder de w heya lwa7daha keda l7d delwa2ty el critical path lesa howa bta3 S_4 el howa $T_{gp} + (N-1)T_{carry} + T_{sum}$ w mafeesh ay improvement 7asal.

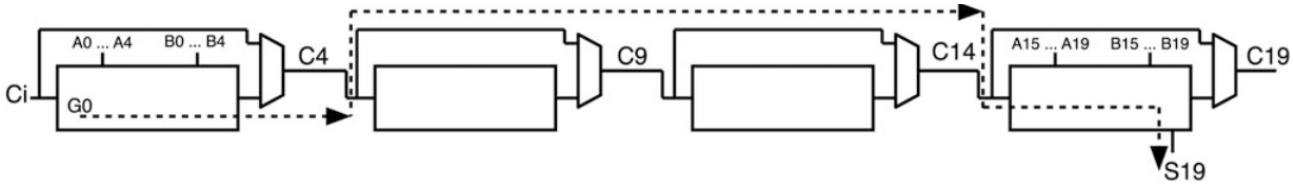
El improvement fel critical path beyebda2 yeban f3ln b2a lama ne build carry bypass adders akbar men carry bypass adders soghayara. Fa assume enena han build 20-bit bypass adder using multiple instances of the previous bypass adder. It will be as shown below:



Fa tefteker eh el critical path hena? akeed le awel wahla han2ool en el critical path hayebda2 men A_0 or B_0, hay3ady through el carry chain bta3t awel block, then through the first mux, then nafs el kalam ma3 el 2 blocks el ba3deya, w a5er block bs hayeb2a bardo el critical path bey3ady through el carry chain kolaha, bs badal ma yo5rog through C_{19} hayo5rog through S_{19} la2en el sum leeh delay akbar men el carry zy ma7na 3arfeen. Fa han2ol en el critical path hena howa:

$$T_{add} = T_{gp} + \left(5T_{carry} + T_{mux}\right) * 3 + \left(4T_{carry} + T_{sum}\right)$$

However, da mesh el critical path. Aw be ma3na tany, m3ak 7a2 f3ln en da akbar delay momken ye7sal fel circuit, bs da mosta7eel eno ye7sal, mafeesh input combinations t5ly el path da y7sl! instead, el critical path el 7a2e2y howa el path shown below, beyebda2 men A_0 or B_0, bey3ady through el carry chain bta3t awel block kolaha, then through the first mux, THEN BEY BYPASS ANY INTERMEDIATE STAGE, then finally through el carry chain bta3t a5er block bs yo5rog through S_{19} mesh C_{19}.



Tyb ezay b2a? actually ana ma2darsh aqne3ak en da el critical path, bs el ana a2dar a3melo howa en a2olak e5tarly ay path tany w ana hasbetlak eno either 3omro ma haye7sal aw en el delay bta3 el path el faat da atwal meno. Fa 5alena na5od el path el badehy el olna 3leeh fel awel w asbetlak eno 3omro ma haye7sal. Watch video 1. Thus, the critical path occurs when all bit positions are propagating except the first bit (and last bit is don't care). It seems strange that a case where everything is propagating would cause the critical path, but any other input would cause a middle bit position to generate, causing the path to start there, which would be shorter.

Thus, the critical path here is:

$$T_{add} = T_{gp} + 5T_{carry} + T_{mux} + (4-2)T_{mux} + 4T_{carry} + T_{sum}$$

In general:

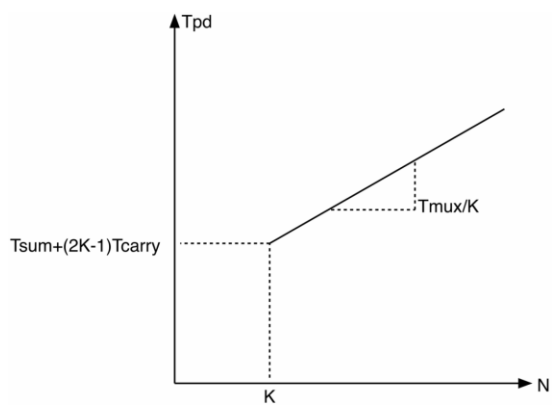
$$T_{add} = T_{gp} + KT_{carry} + T_{mux} + (N/K-2)T_{mux} + (K-1)T_{carry} + T_{sum}$$

$$\therefore \boldsymbol{T_{add} = T_{gp} + KT_{carry} + (N/K-1)T_{mux} + (K-1)T_{carry} + T_{sum}}$$

Where:

- N: total number of FAs.
- K: number of FAs per stage.

Again, the delay scales *linearly* with the number of bits as shown below (neglecting T_{gp}):



However, the slope is now $\frac{T_{mux}}{K}$ instead of T_{carry}. This division by K causes the slope to be less than that in case of ripple carry adder, so the growth in delay in case of carry bypass adder is less than that of a ripple carry adder.

So, what is the optimal value for K?

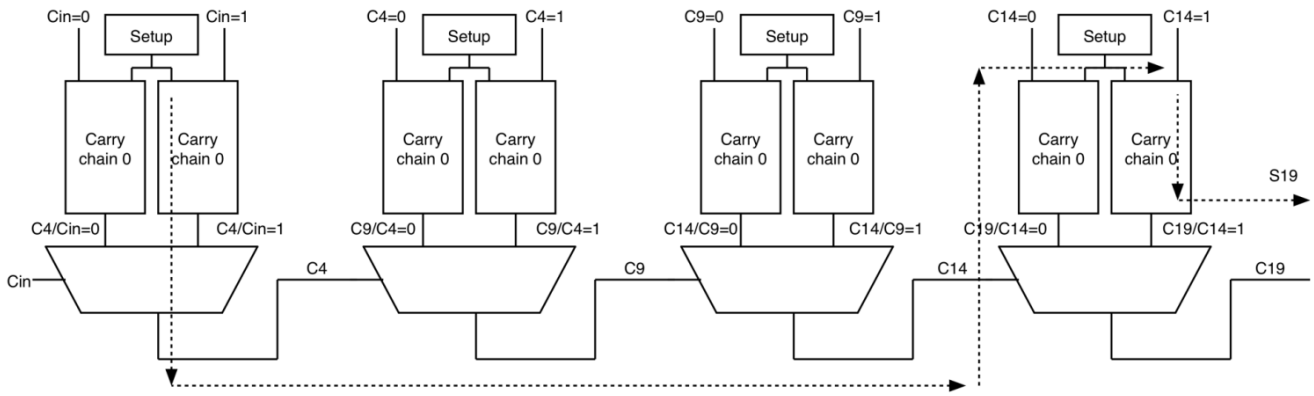
Differentiate T_{add} with respect to K and equate to zero you get:

$$K_{opt.} = \sqrt{\frac{N.T_{mux}}{2T_{carry}}}$$

- It makes sense that K increases as T_{mux} increases la2en lw T_{mux} kebera fa hab2a 3ayez a3ady fi 3adad multiplexers a2al, w bel taly 3ayez number of stages a2al, w bel taly hab2a 3ayez K kebera.
- It makes sense that K decreases as T_{carry} increases la2en lw T_{carry} kebera fa hab2a 3ayez el carry chain el fi awel w a5er stage mayeb2osh kobaar, fa hab2a 3ayez K soghayara.

Carry Select Adder

Fekret-ha heya enaha teda7y bel area moqabel enaha te7asen fel critical path. Ezay? delwa2ty e7na 3arfeen en el beye3melena moshkela howa el carry, sa7? moshkela be ma3na eh? be ma3na en lw 3andy kaza stage wara b3d fa kol stage beteb2a mestaneya el carry out beta3 el stage el ableha 3ashan tetala3 el carry out bet3aha, sa7? helw awy, el nas b2a fakaret fi fekra zakeya gedan w heya enaha t7ot aslan 2 carry chains fi kol stage, b7ees chain te7seb el carry out assuming en el carry in kan 1, wel chain el tanya te7seb el carry out assuming en el carry in kan 0, w sa3et-ha hayeb2a bs door el carry out beta3 el stage el ableya howa eno yegy ye select result men el etneen, da akeed a7san men enena fel awel kona benestana el carry in yegy 3ashan aslan abda2 7esbet el carry out. This is shown below:



Watch video 4.

What is the delay of this adder?

$$T_{add} = T_{gp} + 5T_{carry} + 3T_{mux} + 4T_{carry} + T_{sum}$$

W in general:

$$\boldsymbol{T_{add} = T_{gp} + KT_{carry} + (N/K - 1)T_{mux} + (K - 1)T_{carry} + T_{sum}}$$

Notice en da nafs el delay beta3 carry bypass adder. Tab ezay mafeesh 7aga et7asenet although e7na 3amalna almost doubling lel hardware for the sake of improving the delay? mahowa da besabab restriction maloosh lazma e7na 7ateno 3la nafsena fel architecture el faat da. Eh howa? enena Fardeen en kol el stages fiha el block size equal b3d, el howa K. De mesh fekra zakeya 5ales. Leeh? la2en delwa2ty 3andak stage two masalan betala3 el 2 carries bto3ha $C9/C4 = 0$ and $C9/C4 = 1$ b3d time $T_{gp} + 5T_{carry}$. Tyb enta bet make use of el 2 carries dool emta? lama agy a select benhom using $C4$, el howa lama $C4$ teg-haz y3ny b3d time $T_{gp} + 5_{Tcarry} + T_{mux}$. Wa5ed balak men 7aga? stage 2 el output beta3ha kan gahez, however ana esta5demto T_{mux} later, aw be tare2a tanya stage 2 tala3et el output beta3ha T_{mux} badry. Tab t3ala nkml. Stage 3 hatala3 el 2 carries beto3ha emta? b3d time $T_{gp} + 5T_{carry}$ bardo. Tab wana hasta5demo emta? lama $C9$ yeg-haz, y3ny b3d time $T_{gp} + 5_{Tcarry} + 2T_{mux}$, w bel taly keda stage 3 me5alasa badry be $2T_{mux}$. W hanefdal keda b2a kol ma n5osh fi stages aktar kol ma ala2y el stage bet5alas shoghlaha badry awy. Da akeed waste, ne7elo ezay? enena mane3melsh el stages of equal block size! fa masalan ne5aly stage 1 teb2a K bits, w stage 2 $K + 1$ bits, and so on. Thus, sa3et-ha hayeb2a:

$$N = K + (K + 1) + (K + 2) + \cdots + (K + P - 1)$$

Where P is the number of stages (and also the number of terms in N expression).

$$\therefore N = \frac{P\big((K) + (K + P - 1)\big)}{2} = \frac{P(P + 2K - 1)}{2}$$

if $P \gg K$:

$$\therefore N \approx \frac{P^2}{2} \rightarrow P = \sqrt{2N}$$

After using unequal block sizes, let's now re-write the adder's delay expression:

$$t_{pd} = KT_{carry} + (P - 2)T_{mux} + (K + P - 2)T_{carry} + T_{sum} = KT_{carry} + \big(\sqrt{2N} - 2\big)T_{mux} + \big(K + \sqrt{2N} - 2\big)T_{carry} + T_{sum}$$

See? using unequal block sizes allowed us to reach a sub-linear delay growth instead of linear growth, which is better! Thus, the delay growth of carry-select adders is square root (i.e. $O(\sqrt{N})$).

Carry Lookahead Adder

How should the ideal adder behave? The ideal adder is one in which the delay is independent of the number of bits, N. We have already shown that a ripple carry architecture yields a linear dependence on N. Manipulations of the carry-save adder can yield square root dependence. However, if we dig deeper it looks like we can free the adder delay of all dependence on N.

Listed below are the carries for bit positions 0 through 4

$$
\begin{aligned}
C_0 &= G_0 + P_0\, C_{\text{in}} \\
C_1 &= G_1 + P_1\, C_0 \\
C_2 &= G_2 + P_2\, C_1 \\
C_3 &= G_3 + P_3\, C_2 \\
C_4 &= G_4 + P_4\, C_3
\end{aligned}
$$

The ripple effect is clear in the fact that C_1 needs C_0 as an input, while C_2 needs C_1 and so on. Thus, the carries have to be calculated in series. However, expanding the expression of C_1 in terms of C_0 suggests something a little different:

$$
\begin{aligned}
C_1 &= G_1 + P_1\, C_0 = G_1 + P_1(G_0 + P_0\, C_{\text{in}}) \\
&= G_1 + G_0\, P_1 + P_0\, P_1\, C_{\text{in}}
\end{aligned}
$$

We can expand all the rest of the carries similarly, leading to the following results

$$
\begin{aligned}
C_0 &= G_0 + P_0 C_{\text{in}} \\
C_1 &= G_1 + P_1 G_0 + P_0 P_1 C_{\text{in}} \\
C_2 &= G_2 + P_2 G_1 + P_1 P_2 G_0 + P_0 P_1 P_2 C_{\text{in}} \\
C_3 &= G_3 + P_3 G_2 + P_2 P_3 G_1 + P_1 P_2 P_3 G_0 + P_0 P_1 P_2 P_3 C_{\text{in}} \\
C_4 &= G_4 + P_4 G_3 + P_3 P_4 G_2 + P_2 P_3 P_4 G_1 + P_1 P_2 P_3 P_4 G_0 + P_0 P_1 P_2 P_3 P_4 C_{\text{in}}
\end{aligned}
$$

All carries become dependent only on G's P's and C_{in}. These equations indicate that all carries can start calculating as soon as the operation starts. We do not need to wait for previous carries to calculate the current carry. This suggests that we finally have an adder whose delay is independent of N.

This conclusion is flat out wrong!

Leeh? la2en actually enta lw baseet closely fi el expression bta3 kol carry, hatla2eh implicitly beye7seb kol el carries el ableh. Fa keda el adder mesh independent of N wala 7aga. Bel 3aks, da e7na kaman 3amleen 7araka ghabeya enena mesh benastaghel el 7aga el ma7soba abl keda w bene7sebha tany. Fa masalan C_2 w C_3 w C_4 kolohom beye7sebo C_0 tany. Fa actually e7na hena mesh bs ma3amalnash 7aga, da7na kaman 5alena el delay growth b2a quadratic ma3 N! Fa actually el lookahead adders de leha el worst delay growth ever so far. 5aly balak en da mesh ma3nah enaha malhash este5dam, e7na ben2ol en el delay growth beta3ha w7sh gedan, laken momken 3and 3adad bits olayel (i.e. small N) teb2a a7san men adders tanya with linear delay growth masalan.

Group Generates and Propagates

El zaky b2a fel expanded equations bta3t el carries de mesh enena nroo7 ne implement-ohom zy el hobl keda zy fel lookahead adders, w enama enena ne notice el patterns el fihom el tesa3edna enena f3ln n3ml adders mo7taram gedan:

$$C_0 = G_0 + P_0 C_{in}$$
$$C_1 = G_1 + P_1 G_0 + P_0 P_1 C_{in}$$
$$C_2 = G_2 + P_2 G_1 + P_1 P_2 G_0 + P_0 P_1 P_2 C_{in}$$
$$C_3 = G_3 + P_3 G_2 + P_2 P_3 G_1 + P_1 P_2 P_3 G_0 + P_0 P_1 P_2 P_3 C_{in}$$
$$C_4 = G_4 + P_4 G_3 + P_3 P_4 G_2 + P_2 P_3 P_4 G_1 + P_1 P_2 P_3 P_4 G_0 + P_0 P_1 P_2 P_3 P_4 C_{in}$$

Patterns:

1. The last term in each expression is $P_0 P_1 P_2 \dots C_{in}$. This term without the C_{in} is called "group propagate". It represents all bit positions *propagating* the very first carry in.
2. All the other terms in each expression consists of generate in the current bit position, generate in the previous bit position with the current bit position propagating it, generate in the bit position before the previous one with the current and previous bit positions propagating it, and so on. These terms are called "group generate".

So, each expression consists of "group generate" and "group propagate".

Group generate from bit x to bit y –symbol--> $G_{x:y}$. Example:

$$G_{2:4} = G_4 + G_3 P_4 + G_2 P_3 P_4$$

Group propagate from bit x to bit y –symbol--> $P_{x:y}$. Example:

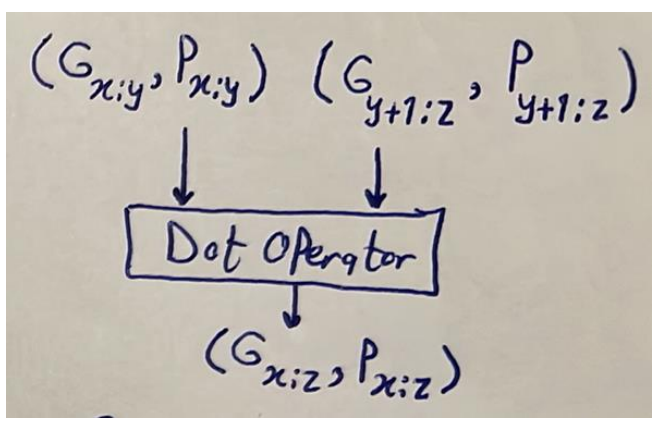
$$P_{2:4} = P_4 P_3 P_2$$

Thus:

$$C_j = G_{0:j} + P_{0:j} C_{in}$$

Dot Operator

3ayzeen ne define operator such that eno ya5od meny 2 groups w yetala3ly group akbar. That is:

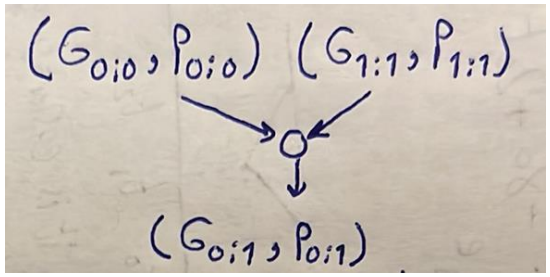


Some properties about the dot operator:

- It can take groups of unequal lengths as inputs. Example: $(G_{1:3}, P_{1:3}) \circ (G_{4:9}, P_{4:9}) -> (G_{1:9}, P_{1:9})$.
- It can operate on overlapping groups. Example: $(G_{1:3}, P_{1:3}) \circ (G_{2:9}, P_{2:9}) -> (G_{1:9}, P_{1:9})$.
- Holes between the two input groups are not allowed. Example: $(G_{1:3}, P_{1:3}) \circ (G_{6:9}, P_{6:9})$ ✗.
- It is not commutative.

What logic equations describe the dot operator?

let's take a simple example to make a generalization from it. Assume we have $(G_{0:0}, P_{0:0})$ and $(G_{1:1}, P_{1:1})$, and we want to dot product them to get $(G_{0:1}, P_{0:1})$. That is:



We know that:

$$G_{0:0} = G_0$$

$$G_{1:1} = G_1$$

$$G_{0:1} = G_1 + G_0 P_1$$

$$P_{0:0} = P_0$$

$$P_{1:1} = P_1$$

$$P_{0:1} = P_1 P_0$$

Thus, it is clear that:

$$G_{0:1} = G_{1:1} + G_{0:0} P_{1:1}$$

$$P_{0:1} = P_{1:1} P_{0:0}$$

Thus, in general:

$$(G_x, P_x) \circ (G_y, P_y) = (G_y + G_x P_y, P_y P_x)$$

where x and y are ranges.

Kogge Stone Adder

It is a type of "parallel prefix adders". In parallel prefix adders, we form large group generates and propagates from smaller group generates and propagates using the dot operator. But why do we want to form large group generates and propagates? la2en lw el groups de et3amalet, fa keda el adder yo3tabar 5allas el operation el matloba meno. Let's take an example. Assume we have a 4-bit adder (i.e. 4-bit inputs). Emta n2ool en el adder de 5alaset? lama yeb2a S_0, S_1, S_2, S_3, C_3 gahzeen (assuming tarteeb el carries howa $C_{in}, C_0, C_1, C_2, C_3$). Tyb ezay a7sebhom kolohom?

$$S_0 = P_0 \oplus C_{in}$$

$$S_1 = P_1 \oplus C_0$$

$$S_2 = P_2 \oplus C_1$$

$$S_3 = P_3 \oplus C_2$$

$$C_3 = G_{0:3} + P_{0:3} C_{in}$$

Tyb w meen C_0, C_1, C_2?

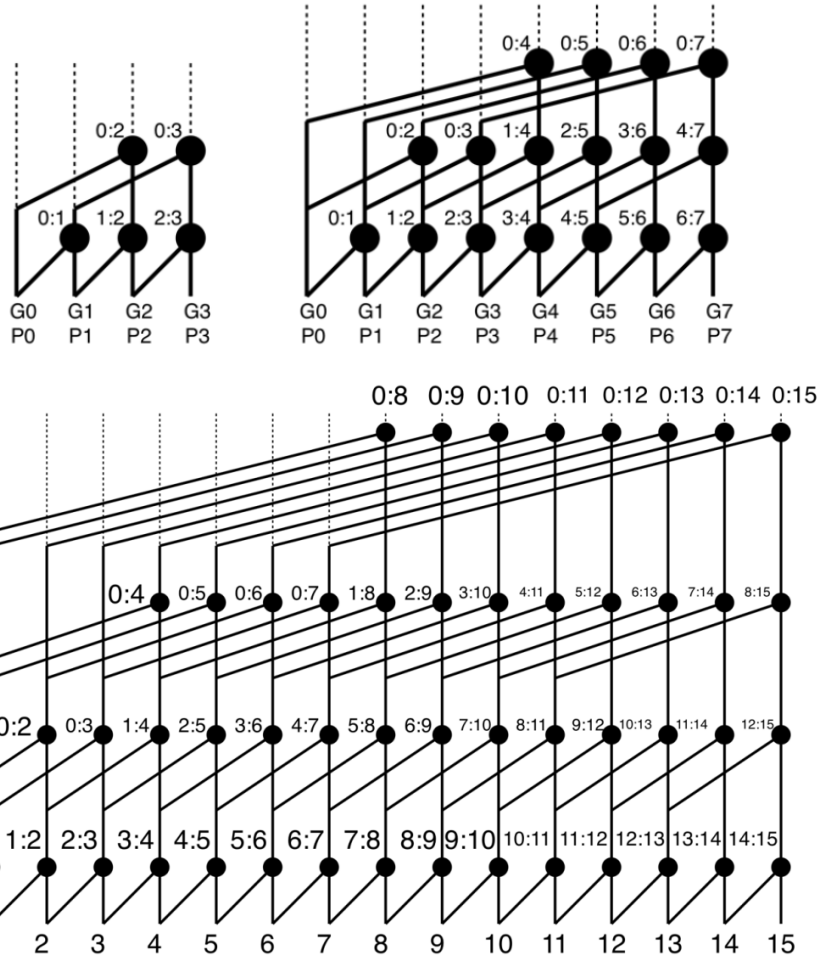
$$C_0 = G_{0:0} + P_{0:0} C_{in}$$

$$C_1 = G_{0:1} + P_{0:1} C_{in}$$

$$C_2 = G_{0:2} + P_{0:2} C_{in}$$

Yb2a men el a5er 3ashan el adder teb2a 5alaset e7na me7tageen $(G_{0:0}, P_{0:0}), (G_{0:1}, P_{0:1}), (G_{0:2}, P_{0:2}), (G_{0:3}, P_{0:3})$ yeb2o gahzeen. Once enohom kolohom ba2o m3ana, Hanroo7 7asbeen $C_0 \rightarrow C_3$ kolohom in parallel, then $S_0 \rightarrow S_3$ kolohom in parallel. Fa 7esbet el sums

wel carries hateb2a baseta, ne7seb bs el group propagates wel group generates aham 7aga. Watch this from 6:00 to 10:11. Now, watch video 2. This type of parallel prefix adders is called “Kogge Stone adders”.



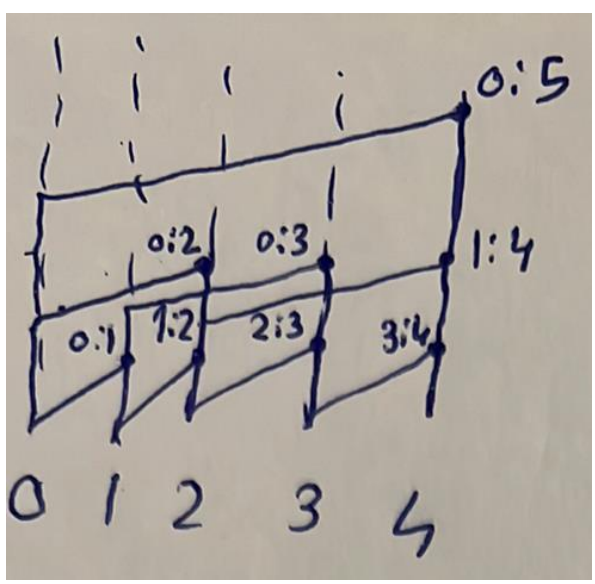
Notice that the critical path in each case is:

$$4 - bit \;\rightarrow\; 2 * T_{dot}$$

$$8 - bit \;\rightarrow\; 3 * T_{dot}$$

$$16 - bit \;\rightarrow\; 4 * T_{dot}$$

So, in general, for an $N - bit$ adder, the critical path is $\log_2(N) * T_{dot}$. Lw el N mesh raqam power of 2, fa just bene3mel ceiling. Fa masalan lw benetkalem 3la 5-bit adder, fa lw garabt tersem el graph el faat da, hatla2y enak 3ashan te7seb $0:5$, me7tag level 3 as shown below:



Fa lw 3ayzeen neb2a more accurate, el mafrood n2ool en el critical path is $\lceil \log_2(N) \rceil * T_{dot}$, where $T_{dot} = T_{and} + T_{or}$.

See? we have finally designed an adder whose delay growth is $O(\log_2(N))$! Notice that logarithmic growth is even better than square root growth. 5aly balak enena ben2ol en el delay growth howa el batee2, bs da mesh ma3nah enaha el delay beta3ha soghayar. In other words, e7na mesh ben2ol en el adders el darasnaha kolaha malhash lazma w en el parallel prefix adder heya a7san 7aga, la2 e7na ben2ol en el parallel prefix adder heya abta2 wa7da men 7ees el delay growth, laken momken gedan 3and 3adad bits olayel (i.e. small N) nela2y en fi adders tanya a7san.

Advantages of Kogge Stone Adder

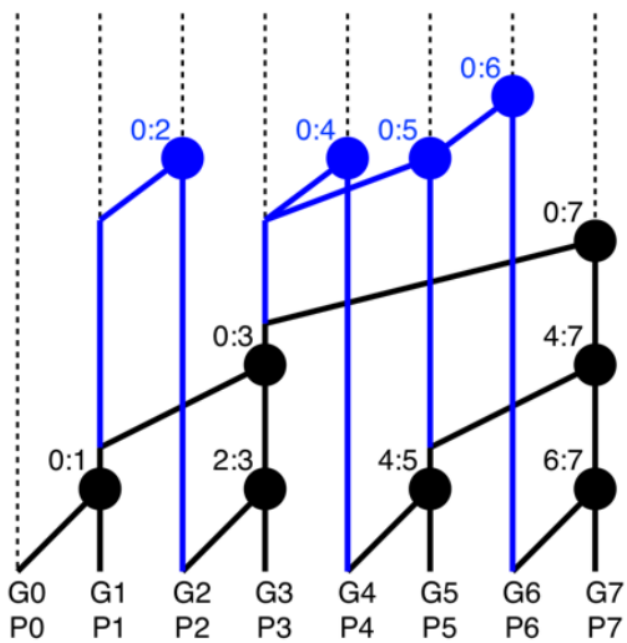
1. Delay growth is $O(\log_2(N))$.
2. Fanout: each dot operator is loaded by at most 2 dot operators. This limited fanout allows for smaller dot operators and better delay.

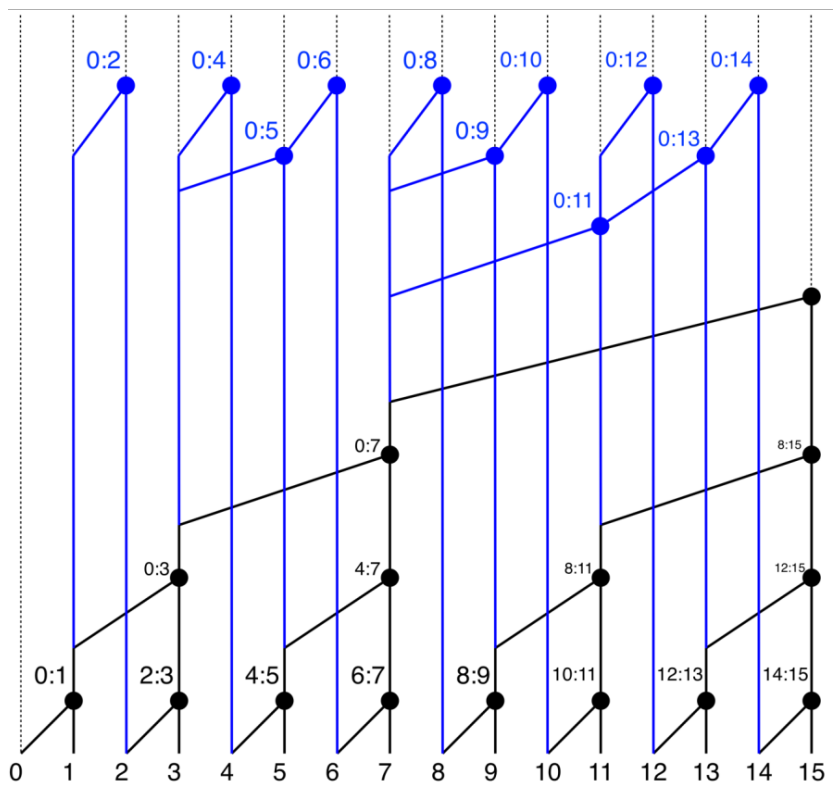
Disadvantages of Kogge Stone Adder

1. Area: fel ripple carry adders, kan 3adad el full adders N, w bel taly el area growth kanet $O(N)$. Fel carry select adders, kan 3adad el carry chains beyeb2a roughly $2N$, fa el area growth kanet $O(2N)$. However, hena b2a fel parallel prefix adders, lw basena masalan 3la $N = 4$, hanla2y enena mesta5demen 5 dot operators, $N = 8$ hanla2ehom 17, $N = 16$ hanla2ehom 49. Fa tab3an bayen gedan en el area growth akbar men $2N$.
2. Huge overhead for placement and routing tools: bos keda 3ala el graph bta3 el 16-bit adder. Shayef kemeyet el routing wel wires el bet3ady men foo2 wires tanya (which requires jumping to different layers)?

Brent Kung Adder

This is another type of parallel prefix adders. Watch video 3.





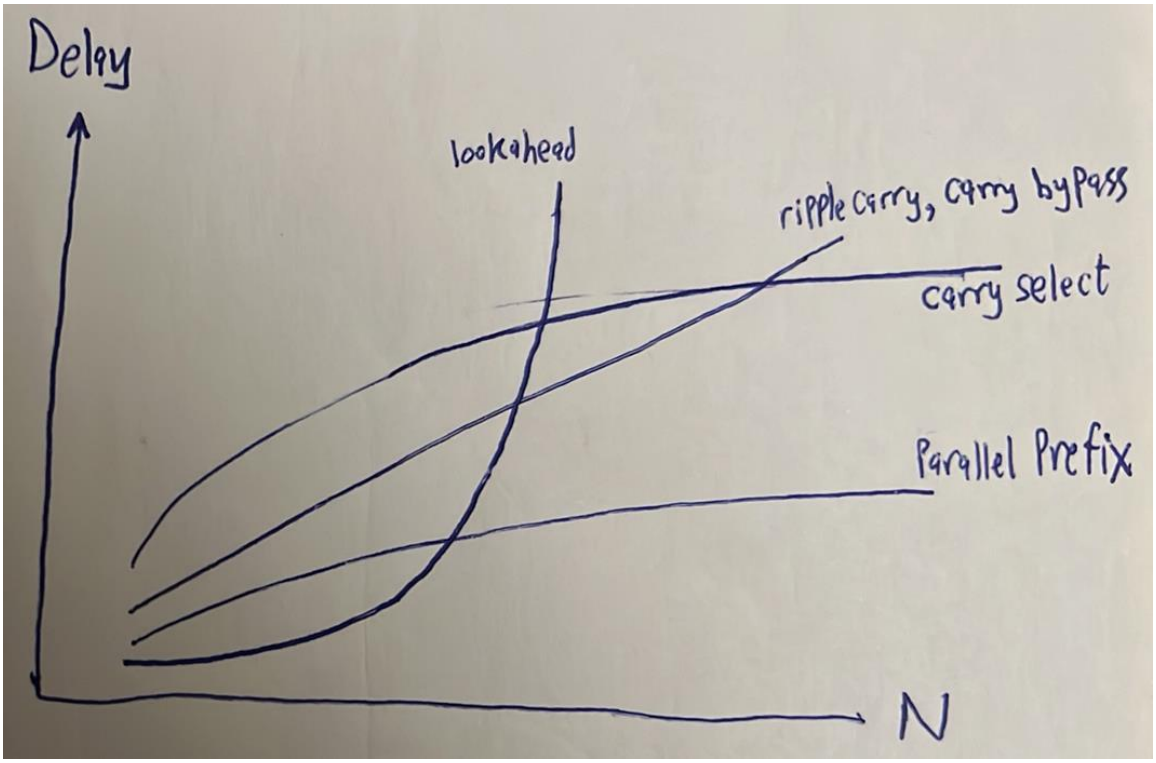
Advantages of Brent Kung Adder

1. Smaller number of dot operators (and hence area) compared to Kogge Stone adders.
2. Less routing overhead compared to Kogge Stone adders.

Disadvantages of Brent Kung Adder

1. Delay growth became $O(\log_2(N + 2))$ instead of $O(\log_2(N))$ (not a big deal).
2. Many dot operators have fanout of more than 2.

Summary


Delay
lookahead
ripple carry, carry bypass
carry select
Parallel Prefix
N